

Mapa de variables de una tarea en SuperCompressARC

Objetivo

Este documento sigue una sola tarea desde el JSON hasta las soluciones finales. Ordena las variables por el momento en que se crean y muestra:

- quien crea cada variable;
- que contiene;
- su tipo y forma;
- si persiste o se recalcula;
- que variable la consume despues.

El recorrido de referencia es el de `analyze_example.py` para la tarea `007bbfb7`, pero las relaciones son las mismas para cualquier tarea.

Leyenda

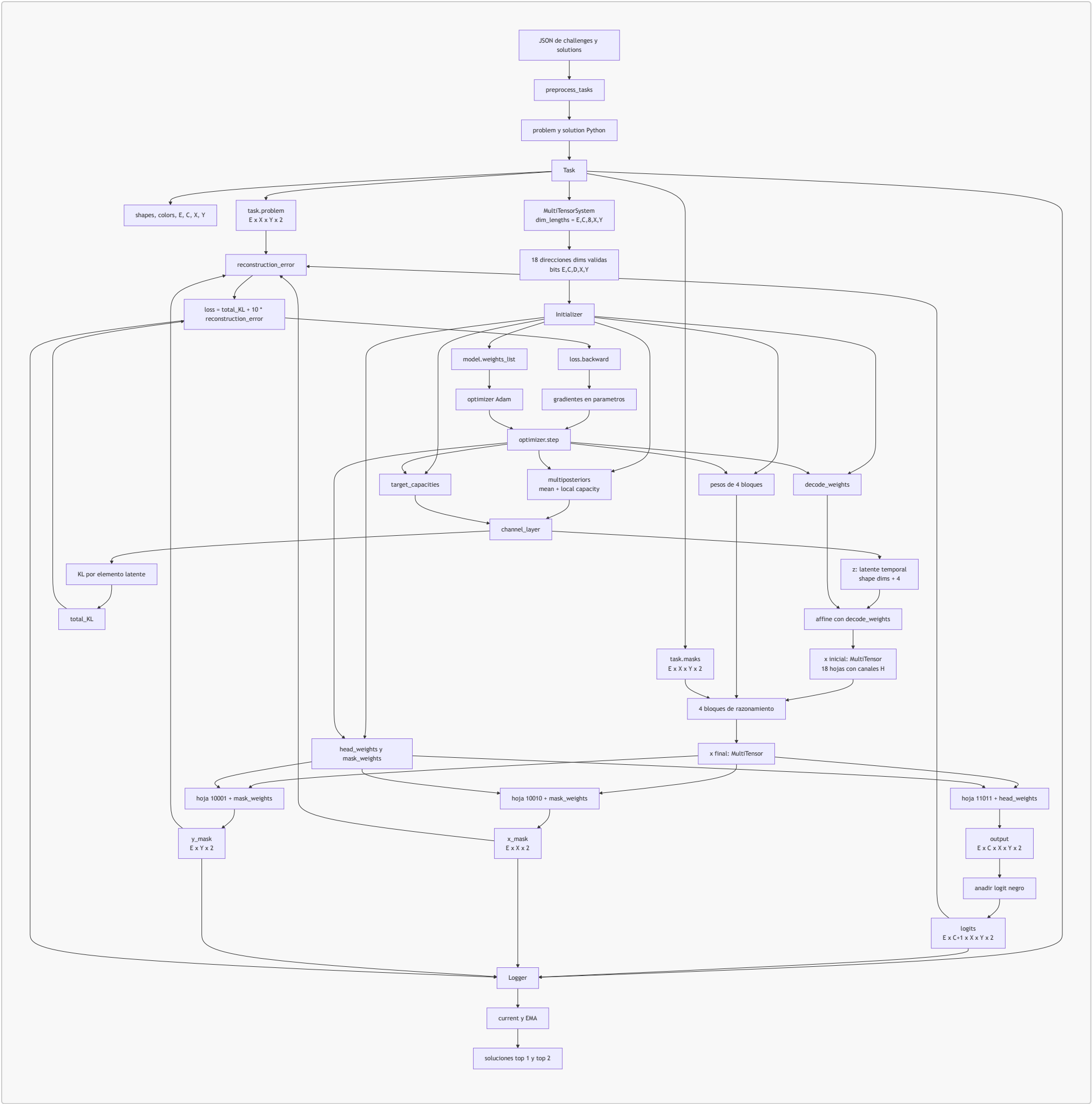
Usaremos estos nombres para las longitudes dependientes de la tarea:

Símbolo	Significado	Variable
E	Numero total de ejemplos	<code>task.n_examples</code>
T	Numero de ejemplos de test	<code>task.n_test</code>
C	Numero de colores no negros	<code>task.n_colors</code>
D	Numero fijo de direcciones	<code>8</code>
X	Maximo numero de filas	<code>task.n_x</code>
Y	Maximo numero de columnas	<code>task.n_y</code>
L	Canales latentes	<code>decoding_dim = 4</code>
H	Canales del residual	<code>16</code> sin direccion, <code>8</code> con direccion

Las variables se clasifican en cuatro familias:

Familia	Marca	Persiste entre pasos	Adam la modifica
Dato observado	DATO	Si	No
Parametro del modelo	PARAMETRO	Si	Si, si participa en la perdida
Activacion temporal	ACTIVACION	No	No directamente
Estado de seguimiento	ESTADO	Si	No

Mapa general



La flecha de `optimizer.step` de vuelta a los parametros representa el ciclo de entrenamiento: el paso siguiente vuelve a crear `z`, `x` y `logits` usando los parametros ya modificados.

Fase 1: variables del programa conductor

`analyze_example.py` crea primero las opciones de ejecucion:

Orden	Variable	Tipo	Contenido	Consumidor
1	<code>args.accel_preset</code>	<code>str</code>	Preset <code>baseline</code> , <code>bf16</code> , <code>compile</code> o <code>full</code>	<code>accel.config_from_preset</code>
2	<code>args.inductor_cache_dir</code>	<code>str</code>	Cache dedicada; por defecto <code>/mnt/supercompressarc-cache/.inductor_cache</code>	Presets compilados
3	<code>accel_cfg</code>	<code>AccelConfig</code>	Configuracion efectiva del proceso y del <code>forward</code>	<code>accel.configure_process</code> , <code>accel.apply</code>
4	<code>effective_cache_dir</code>	<code>str</code> <code>None</code>	<code>TORCHINDUCTOR_CACHE_DIR</code> efectivo tras aplicar precedencias	Log y CSV
5	<code>run_label</code>	<code>str</code>	<code>--run-label</code> o, por defecto, el preset	Nombre del CSV de tiempos
6	<code>split</code>	<code>str</code>	<code>training</code> , <code>evaluation</code> o <code>test</code>	<code>preprocess_tasks</code>

Orden	Variable	Tipo	Contenido	Consumidor
7	<code>task_name</code>	<code>str</code>	Identificador de la tarea	Carga del JSON y nombres de archivos
8	<code>n_iterations</code>	<code>int</code>	Pasos solicitados por <code>--iterations</code>	Bucle de entrenamiento
9	<code>folder</code>	<code>str</code>	<code>results/<task_name>/</code>	Graficas, NPZ y CSV

Todavia no existe ningun tensor neuronal.

Fase 2: carga de los JSON

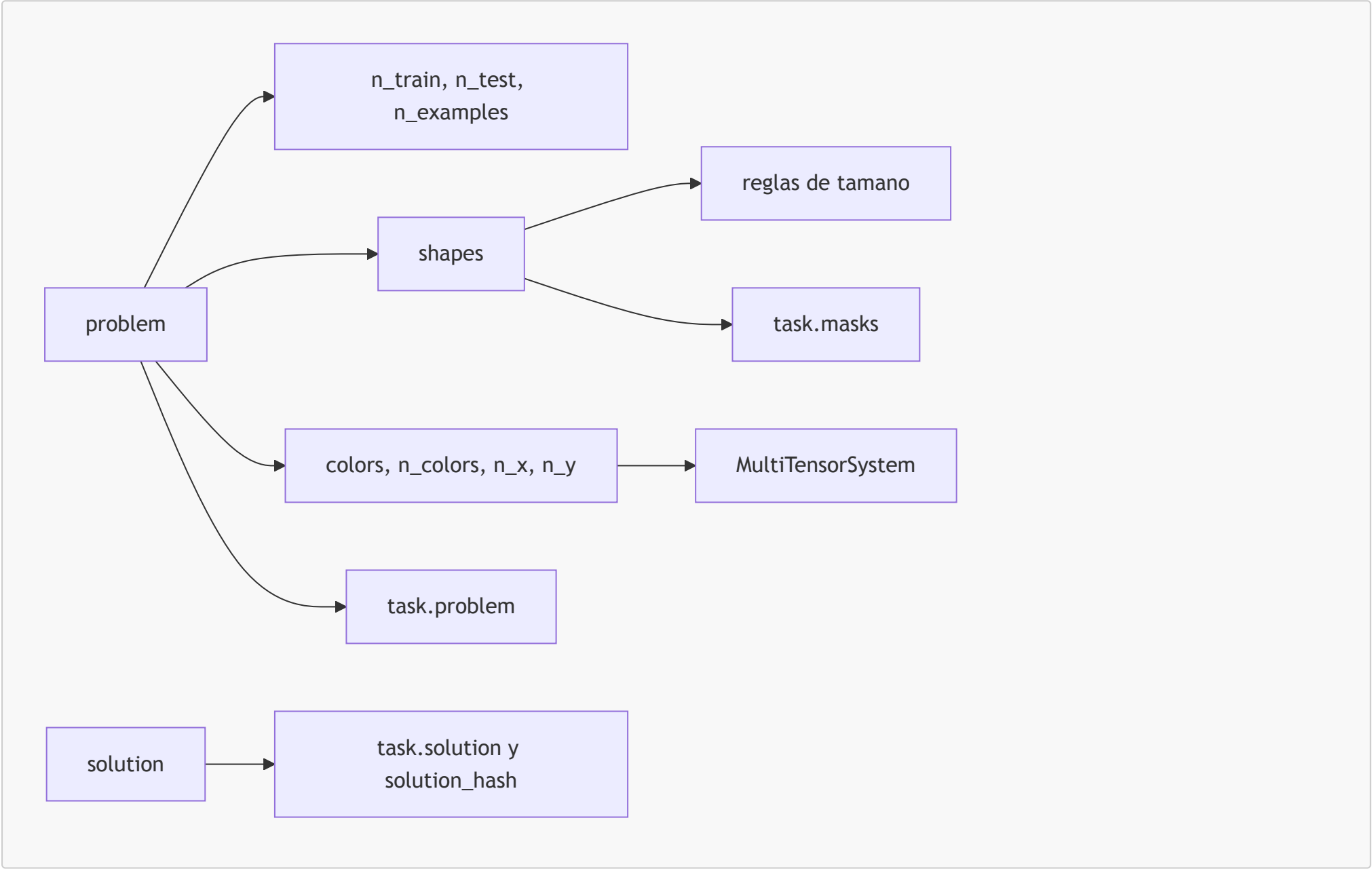
`preprocess_tasks(split, [task_name])` crea:

Variable	Tipo	Contenido
<code>problems</code>	<code>dict</code>	Todas las tareas del archivo <code>challenges.json</code>
<code>solutions</code>	<code>dict</code> o <code>None</code>	Soluciones verdaderas del split, si existen
<code>task_names</code>	<code>list[str]</code>	Claves de <code>problems</code>
<code>problem</code>	<code>dict</code>	Una sola tarea, con listas <code>train</code> y <code>test</code>
<code>solution</code>	Lista o <code>None</code>	Outputs verdaderos de test para evaluar

`problem` conserva las cuadrículas tal como aparecen en JSON. Estos valores son datos, no parametros entrenables.

Fase 3: construccion de `Task`

El constructor crea sus atributos en este orden:



3.1 Identidad y cantidades

Variable	Familia	Tipo	Significado
<code>task.task_name</code>	DATO	<code>str</code>	Identificador de la tarea
<code>task.n_train</code>	DATO	<code>int</code>	Demostraciones con input y output
<code>task.n_test</code>	DATO	<code>int</code>	Ejemplos cuyo output se predice
<code>task.n_examples</code>	DATO	<code>int</code>	<code>n_train + n_test</code>
<code>task.unprocessed_problem</code>	DATO	<code>dict</code>	Referencia al problema JSON original

3.2 Formas reales

`task.shapes` es una lista con esta estructura:

```
task.shapes[example_num][in_out_mode][axis]
                        E      0/1      0/1
```

Indice	Valor
<code>in_out_mode = 0</code>	Input
<code>in_out_mode = 1</code>	Output
<code>axis = 0</code>	Filas, eje <code>x</code> en el codigo
<code>axis = 1</code>	Columnas, eje <code>y</code> en el codigo

Las variables `in_out_same_size`, `all_in_same_size` y `all_out_same_size` resumen regularidades de tamaño. Tambien permiten inferir el tamaño esperado del output de test.

3.3 Colores y espacio rectangular

Variable	Tipo	Significado
<code>task.colors</code>	<code>list[int]</code>	Colores ARC presentes, incluido el negro <code>0</code>
<code>task.n_colors</code>	<code>int</code>	Colores no negros; longitud del eje <code>C</code> del multitensor
<code>task.n_x</code>	<code>int</code>	Maximo de filas reservado
<code>task.n_y</code>	<code>int</code>	Maximo de columnas reservado

Todas las cuadrículas se alojan en un rectangulo comun `X x Y`. Las celdas que sobran son padding.

3.4 `task.masks`: mascara espacial

```
tipo      = torch.Tensor float32
forma     = [E, X, Y, 2]
valores   = 1 para celda real, 0 para padding
familia   = DATO
```

Esta mascara se calcula con las formas reales. La consumen `cummax`, `shift` y algunas reducciones de `share_down` para no tratar el padding como contenido.

3.5 `task.problem`: colores observados

Se construye primero en one-hot:

```
numpy.ndarray [E, C+1, X, Y, 2]
```

Despues `argmax(axis=1)` elimina el eje one-hot:

```
torch.Tensor entero [E, X, Y, 2]
```

Cada celda contiene un indice interno de `task.colors`. Es el objetivo de la reconstruccion; no es una entrada pasada a `model.forward()`.

3.6 `task.solution`

```
task.solution      Tensor [T, X, Y] o None
task.solution_hash int o None
```

Solo se emplean para evaluar la respuesta. No participan en el entrenamiento.

Fase 4: `MultiTensorSystem` y `dims`

`Task` crea:

```
task.multitensor_system.dim_lengths = [E, C, 8, X, Y]
                                     E  C  D  X  Y
```

Una variable `dims` contiene cinco bits:

```
[example, color, direction, x, y]
```

Ejemplo:

```
dims = [1, 1, 0, 1, 1]
```

No contiene datos ni tamaños. Indica que una representacion distingue ejemplo, color, fila y columna, pero no direccion.

`MultiTensorSystem` examina las 32 combinaciones y conserva exactamente 18 validas. Cada `dims` valida es una direccion dentro del contenedor `MultiTensor` y selecciona una hoja.

Dos mascarar diferentes

Nombre	Forma	Funcion
<code>dims</code>	Lista de 5 bits	Elegir los ejes de una hoja del multitensor
<code>task.masks</code>	<code>[E,X,Y,2]</code>	Marcar celdas reales frente a padding

No existe una unica mascara binaria de cinco dimensiones aplicada al problema. Existen 18 listas `dims` validas que describen 18 vistas, y por separado existe la mascara espacial `task.masks`.

Fase 5: construccion del modelo

`model = ARCCompressor(task)` crea un modelo nuevo y especifico para la tarea.

5.1 Tamaños fijos de arquitectura

Variable de clase	Valor	Funcion
<code>n_layers</code>	4	Numero de bloques de razonamiento
<code>decoding_dim</code>	4	Canales del latente <code>z</code>
<code>share_up_dim</code>	16	Ancho interno de <code>share_up</code>
<code>share_down_dim</code>	8	Ancho interno de <code>share_down</code>
<code>softmax_dim</code>	2	Ancho de entrada de la operacion softmax
<code>cummax_dim</code>	4	Ancho interno de cummax
<code>shift_dim</code>	4	Ancho interno de shift
<code>nonlinear_dim</code>	16	Ancho interno de SiLU

`channel_dim_fn(dims)` decide el ancho estable de las hojas de `x`:

```
dims[2] = 0, sin eje direccion -> H = 16 canales
dims[2] = 1, con eje direccion -> H = 8 canales por direccion
```

5.2 `Initializer`

El objeto temporal `initializer` conserva:

```
initializer.multitensor_system
initializer.channel_dim_fn
initializer.weights_list = []
```

Cada parametro creado se añade a `weights_list`.

Inicializacion lineal

```
[weight, bias]
weight = randn(n_in, n_out) / sqrt(n_in)
bias    = randn(n_out) / sqrt(n_in)
```

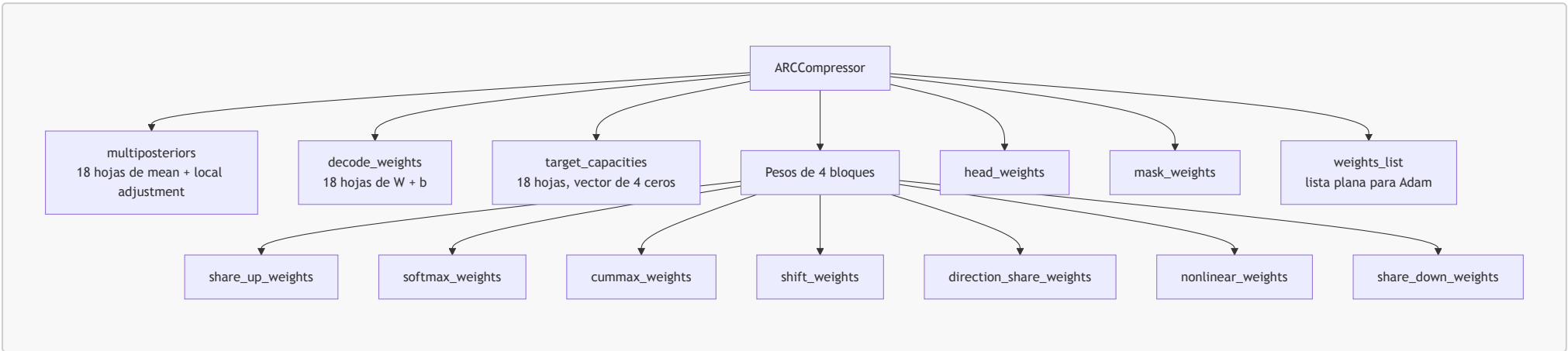
Inicializacion posterior

Para cada una de las 18 hojas:

```
mean                = 0.01 * randn(shape(dims, 4))
local_capacity_adjustment = zeros(shape(dims, 4))
posterior           = [mean, local_capacity_adjustment]
```

Ambos son PARAMETROS. **mean** comienza como ruido normal pequeño; sus ejes los determinan **dims** y sus ultimos cuatro valores son canales latentes.

5.3 Parametros persistentes del modelo



Atributo	Estructura	Funcion
<code>multiposteriors</code>	<code>MultiTensor[[mean, local_adjustment]]</code>	Memoria latente entrenable
<code>decode_weights</code>	<code>MultiTensor[[W,b]]</code>	Convertir 4 canales latentes en 16 u 8 canales
<code>target_capacities</code>	<code>MultiTensor[Tensor[4]]</code>	Regular la capacidad informativa de <code>z</code>
<code>*_weights[layer]</code>	Lista de 4 <code>MultiTensor</code>	Transformar y comunicar <code>x</code>
<code>head_weights</code>	<code>[W,b]</code>	Convertir 16 rasgos en logits input/output
<code>mask_weights</code>	<code>[W,b]</code>	Producir puntuaciones de filas y columnas
<code>weights_list</code>	<code>list[Tensor]</code>	Referencias planas que recibe Adam

Un bloque residual almacena dos lineales:

```
[[W_down, b_down], [W_up, b_up]]
```

`direction_share_weights` es distinto: cada hoja contiene una tabla `8 x 8` de pares `[W,b]`, uno por combinacion direccion destino/origen.

Fase 6: optimizador y logger

Antes del primer paso se crean:

```
optimizer = torch.optim.Adam(
    model.weights_list,
    lr=0.01,
    betas=(0.5, 0.9),
)
train_history_logger = Logger(task)
```

`optimizer` no contiene las activaciones `z` o `x`. Contiene referencias a los PARAMETROS de `weights_list` y, tras el primer paso, crea para cada parametro activo sus momentos `exp_avg` y `exp_avg_sq`.

El `Logger` crea curvas vacias, buffers de logits actuales y medias exponenciales para los ejemplos de test.

Fase 7: una iteracion de entrenamiento

El bucle llama 2000 veces a:


```
train.take_step(task, model, optimizer, train_step, train_history_logger)
```

7.1 Limpiar gradientes anteriores

```
optimizer.zero_grad()
```

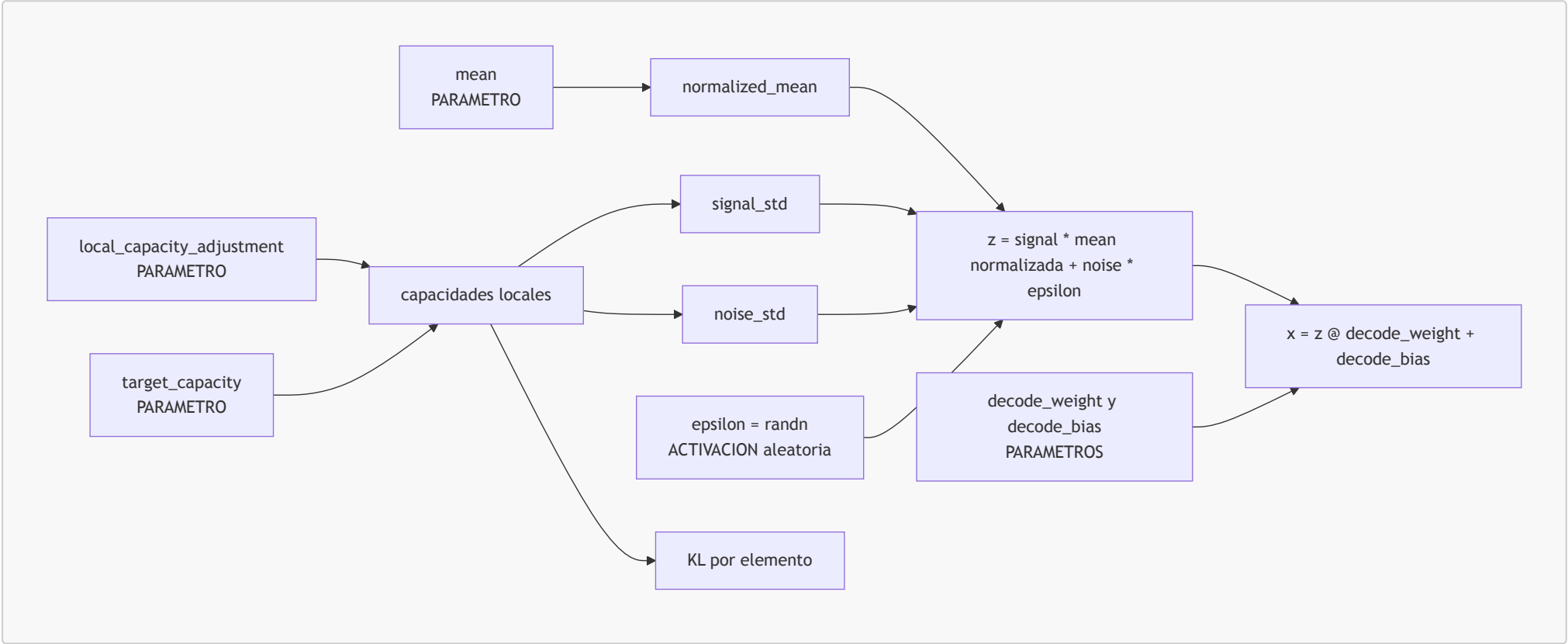
Esto limpia `.grad`. No cambia los parametros ni el estado de Adam.

7.2 `decode_latents`: crear `z`, KL y `x`

`model.forward()` empieza con:

```
x, KL_amounts, KL_names = layers.decode_latents(
    model.target_capacities,
    model.decode_weights,
    model.multiposteriors,
)
```

`multify` repite el siguiente recorrido para cada una de las 18 hojas.



Variables de `channel_layer`

Orden	Variable	Familia	Forma por hoja	Funcion
1	<code>mean</code>	PARAMETRO	<code>shape(dims,4)</code>	Señal latente aprendida
2	<code>local_capacity_adjustment</code>	PARAMETRO	<code>shape(dims,4)</code>	Distribuir capacidad local
3	<code>target_capacity</code>	PARAMETRO	<code>[4]</code>	Capacidad global por canal latente
4	<code>desired_global_capacity</code>	ACTIVACION	<code>[4]</code>	Capacidad positiva transformada
5	<code>output_scaling</code>	ACTIVACION	<code>[4]</code>	Escala final de <code>z</code>
6	<code>desired_local_capacity</code>	ACTIVACION	<code>shape(dims,4)</code>	Capacidad de cada elemento
7	<code>noise_std, noise_var</code>	ACTIVACION	<code>shape(dims,4)</code>	Ruido permitido
8	<code>signal_std, signal_var</code>	ACTIVACION	<code>shape(dims,4)</code>	Señal permitida
9	<code>normalized_mean</code>	ACTIVACION	<code>shape(dims,4)</code>	<code>mean</code> centrada y escalada
10	<code>z</code>	ACTIVACION	<code>shape(dims,4)</code>	Muestra latente temporal
11	<code>KL</code>	ACTIVACION	<code>shape(dims,4)</code>	Coste de informacion por elemento

`z` no persiste. Se vuelve a muestrear en cada `forward()`.

Decodificar `z` para obtener `x`

```
z [...,4] @ decode_weight [4,H] + decode_bias [H] = x [...,H]
```

`decode_latents` devuelve:

Variable	Tipo	Contenido
<code>x</code>	<code>MultiTensor[Tensor]</code>	18 hojas de activaciones con canales <code>H</code>
<code>KL_amounts</code>	<code>list[Tensor]</code>	Un tensor KL por hoja
<code>KL_names</code>	<code>list[str]</code>	El <code>dims</code> correspondiente a cada KL

Ejemplo sin direccion:

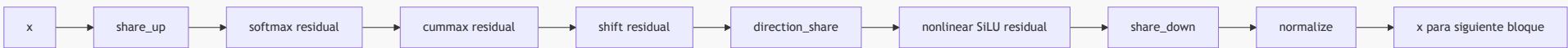
```
dims = [1,1,0,1,1]
x[dims].shape = [E,C,X,Y,16]
```

Ejemplo con direccion:

```
dims = [1,1,1,1,1]
x[dims].shape = [E,C,8,X,Y,8]
```

Una posicion y una direccion concretas contienen 8 canales. Si se conservan las ocho direcciones, la estructura local es una matriz `[8,8]`, no un unico vector de 64 canales.

7.3 Cuatro bloques modifican `x`



Operacion	Variables que consume	Efecto sobre <code>x</code>
<code>share_up</code>	Todas las hojas y <code>share_up_weights</code>	Lleva vistas generales a vistas detalladas
<code>softmax</code>	Cada hoja y <code>softmax_weights</code>	Compara subconjuntos de ejes
<code>cummax</code>	Hojas direccionales, <code>task.masks</code> , pesos	Propaga maximos espaciales
<code>shift</code>	Hojas direccionales, <code>task.masks</code> , pesos	Desplaza informacion entre vecinos
<code>direction_share</code>	Ocho direcciones y pesos <code>8 x 8</code>	Mezcla informacion entre direcciones
<code>nonlinear</code>	Cada hoja y pesos	Introduce no linealidad SiLU
<code>share_down</code>	Todas las hojas, mascaras y pesos	Resume vistas detalladas
<code>normalize</code>	Cada hoja	Controla media y escala

Los bloques residuales calculan conceptualmente:

```
delta = W_up(operacion(W_down(x)))
x_nuevo = x_anterior + delta
```

La forma estable de cada hoja sigue terminando en `H`; cambian sus valores.

7.4 Cabezas de salida

Las cabezas no consumen las 18 hojas directamente. Seleccionan tres:

Hoja	Forma antes de la cabeza	Cabeza	Salida
<code>x[[1,1,0,1,1]]</code>	<code>[E,C,X,Y,16]</code>	<code>head_weights [16,2]</code>	<code>output [E,C,X,Y,2]</code>
<code>x[[1,0,0,1,0]]</code>	<code>[E,X,16]</code>	<code>mask_weights [16,2]</code>	<code>x_mask [E,X,2]</code>
<code>x[[1,0,0,0,1]]</code>	<code>[E,Y,16]</code>	<code>mask_weights [16,2]</code>	<code>y_mask [E,Y,2]</code>

El ultimo eje de tamaño 2 representa modo input/output. `output` contiene solo los `C` colores no negros. `take_step` antepone un plano fijo de ceros para el negro:


```
output [E,C,X,Y,2] -> logits [E,C+1,X,Y,2]
```

7.5 Calculo de la perdida

Primero se suman todos los elementos de `KL_amounts`:

```
total_KL = escalar
```

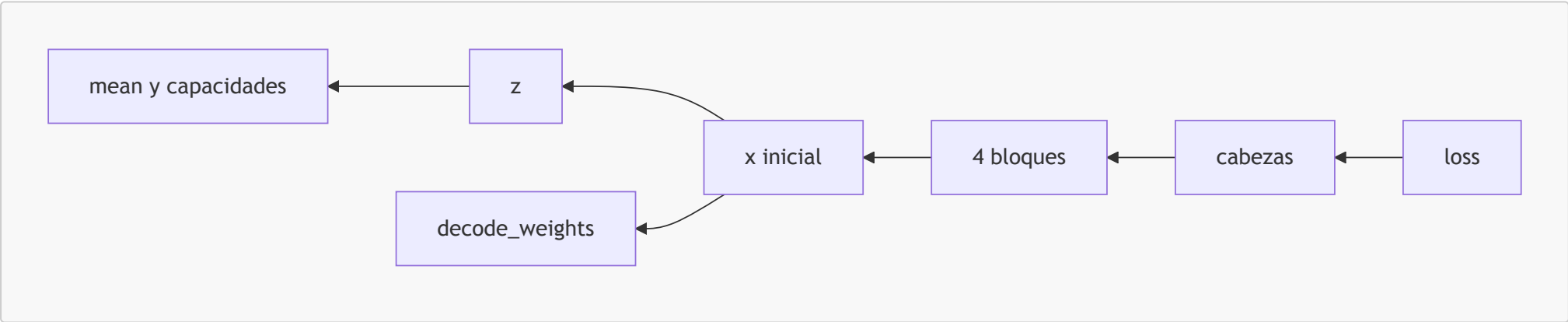
Despues, para cada ejemplo conocido y modo, se crean:

Variable	Forma	Procedencia
<code>logits_slice</code>	<code>[C+1,X,Y]</code>	Corte de <code>logits</code>
<code>problem_slice</code>	<code>[X,Y]</code>	Corte de <code>task.problem</code>
<code>output_shape</code>	<code>[2]</code>	<code>task.shapes</code>
<code>x_logprobs</code>	<code>[n_x_offsets]</code>	<code>x_mask</code>
<code>y_logprobs</code>	<code>[n_y_offsets]</code>	<code>y_mask</code>
<code>target_crop</code>	<code>[out_x,out_y]</code>	Zona real de <code>problem_slice</code>
<code>logits_crops</code>	<code>[n_offsets,C+1,out_x,out_y]</code>	Todas las ventanas candidatas
<code>target_batch</code>	<code>[n_offsets,out_x,out_y]</code>	Objetivo repetido
<code>ce</code>	<code>[n_offsets,out_x,out_y]</code>	Error por pixel y ventana
<code>ce_sum</code>	<code>[n_x_offsets,n_y_offsets]</code>	Error de color por ventana
<code>logprobs</code>	<code>[n_x_offsets,n_y_offsets]</code>	Evidencia de forma, posicion y color
<code>logprob</code>	Escalar	Evidencia agregada

La suma de `-logprob` produce `reconstruction_error`.

$$loss = total_KL + 10 \cdot reconstruction_error$$

7.6 Retropropagacion y actualizacion



`loss.backward()` no modifica nada. Calcula `.grad` para cada PARAMETRO usado.

`optimizer.step()` modifica los parametros usando esos gradientes y el estado de Adam. Por ejemplo:

```
mean_nueva = mean_anterior - correccion_Adam
```

En el paso siguiente:

```
parametros modificados
-> nuevo z aleatorio condicionado por mean
-> nuevo x
-> nuevos logits
-> nueva loss
```

Los valores de `z` y `x` cambian, pero no porque Adam los almacene y edite. Cambian porque son ACTIVACIONES recalculadas a partir de PARAMETROS nuevos.

Fase 8: registro y seleccion de soluciones

El logger recibe tensores con `.detach()` , por lo que ya no participan en los gradientes.

Variable de estado	Forma o tipo	Funcion
<code>KL_curves</code>	<code>dict[str, list]</code>	KL escalar de cada hoja por paso
<code>total_KL_curve</code>	Lista	KL total por paso
<code>reconstruction_error_curve</code>	Lista	Error de reconstruccion por paso
<code>loss_curve</code>	Lista	Perdida por paso
<code>current_logits</code>	<code>[T, C+1, X, Y]</code>	Prediccion test actual
<code>current_x_mask</code>	<code>[T, X]</code>	Scores actuales de filas
<code>current_y_mask</code>	<code>[T, Y]</code>	Scores actuales de columnas
<code>ema_logits</code>	<code>[T, C+1, X, Y]</code>	Media exponencial de predicciones
<code>ema_x_mask</code>	<code>[T, X]</code>	Media exponencial de filas
<code>ema_y_mask</code>	<code>[T, Y]</code>	Media exponencial de columnas
<code>solution_hashes_count</code>	<code>dict[int, float]</code>	Evidencia acumulada por candidata
<code>solution_most_frequent</code>	Tupla	Primer intento pass@2
<code>solution_second_most_frequent</code>	Tupla	Segundo intento pass@2

La conversion de logits a una candidata sigue:

```
logits
-> argmax de color
-> mejor recorte segun x_mask e y_mask
-> indices internos convertidos con task.colors
-> tupla de cuadrículas
-> hash y score
-> actualizar top 1 y top 2
```

Fase 9: analisis posterior de `analyze_example.py`

Una vez terminados los 2000 pasos se guardan:

Variable	Archivo o estructura	Contenido
<code>step_profile</code>	CSV	Tiempo wall y CPU por paso
<code>KL_curves</code>	NPZ	Evolucion del KL de cada hoja
<code>reconstruction_error_curve</code>	NPZ	Evolucion del error
<code>multiposteriors</code>	NPZ	<code>mean</code> y ajustes ya entrenados
<code>target_capacities</code>	NPZ	Capacidades ya entrenadas
<code>decode_weights</code>	NPZ	Pesos de decodificacion entrenados

El script vuelve a llamar 100 veces a `decode_latents`:

```
samples = 100 MultiTensor x muestreados
means    = promedio de esos samples
```

En esta parte `means` no es el PARAMETRO `mean` del posterior. Es una variable de analisis que promedia activaciones `x` para reducir el ruido de muestreo. Despues SVD obtiene componentes principales para visualizar los patrones aprendidos en las hojas con KL significativo.

Variables con nombres faciles de confundir

Nombre parecido	Significado real
<code>dims</code>	Cinco bits que seleccionan ejes de una hoja
<code>task.masks</code>	Mascara espacial de celdas reales y padding
<code>x_mask, y_mask</code>	Scores aprendidos para recortar la prediccion
<code>mean en posterior</code>	PARAMETRO latente entrenable

Nombre parecido	Significado real
mean en average_samples	Promedio temporal de muestras x para graficar
z en channel_layer	Latente estocastico de cuatro canales
z en add_residual	Variable local temporal que representa delta
x	MultiTensor de activaciones del flujo residual
output	Logits de colores no negros al salir de forward
logits	output despues de añadir el plano negro
task.solution	Verdad de test usada solo para evaluar
solution_most_frequent	Prediccion seleccionada por el logger

Linea temporal resumida

```
UNA VEZ POR TAREA
JSON
-> problem, solution
-> Task
-> MultiTensorSystem
-> parametros del modelo
-> optimizer y Logger

2000 VECES
parametros
-> z
-> x inicial
-> x tras cuatro bloques
-> output, x_mask, y_mask
-> logits
-> total_KL + reconstruction_error
-> loss
-> gradientes
-> parametros actualizados
-> registro y candidatas

AL FINAL
curvas + pesos aprendidos + 100 muestras
-> promedio
-> SVD
-> graficas de componentes latentes
```

Preguntas para identificar cualquier variable

Al detenerse en el debugger, conviene responder siempre en este orden:

1. ¿Es DATO, PARAMETRO, ACTIVACION o ESTADO?
2. ¿Quien la crea?
3. ¿Que significa cada eje de su forma?
4. ¿Persiste hasta el paso siguiente?
5. ¿Quien la consume?
6. ¿Recibe cambios de Adam o se recalcula desde otros valores?

Con esas seis preguntas se puede ubicar cualquier nombre del flujo sin confundir la estructura del problema con la representacion aprendida.